Labo 10 — Questions

Question 1 [Diagnostic]

Un serveur de recette n'a plus d'espace disque. `podman system df` est propre : peu d'images, aucun volume orphelin. Pourtant `du -sh ~/.local/share/containers/storage/overlay-containers/*/userdata/` montre un conteneur à 40 Go. Qu'est-ce qui remplit le disque, pourquoi `system df` ne le voit-il pas dans la colonne « reclaimable », et quelles sont les deux corrections (immédiate et durable) ?

Question 2 [Prédiction]

```
podman run -d --name api --memory 256m -e JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=90" api:1.0
```

L'API fonctionne pendant dix minutes, puis le conteneur disparaît de `podman ps` ; `podman ps -a` le montre `Exited (137)`. Les logs de l'application ne contiennent aucune `OutOfMemoryError`. Expliquez la séquence exacte, dites quelle commande confirme le diagnostic, et proposez deux corrections.

Question 3 [Compréhension]

Que montrent respectivement `podman stats` et `podman top` pour un conteneur, en quoi diffèrent-ils de `podman inspect`, et pourquoi `podman stats` peut-il afficher une **limite** de 32 Go pour un conteneur lancé sans `--memory` sur une machine de 32 Go ?

Question 4 [Diagnostic]

```
FROM localhost/api:1.0
HEALTHCHECK --interval=5s --retries=3 CMD wget -qO- http://localhost:8080/actuator/health ||
exit 1
```

```
$ podman build -t api:1.1 .
$ podman run -d --name api api:1.1
$ podman healthcheck run api
Error: container … has no defined healthcheck
```

Le Dockerfile est correct. Pourquoi Podman ne voit-il pas le healthcheck, comment le vérifier sur l'image, et quelle option corrige le build ? Que se serait-il passé avec Docker ?

Question 5 [Analyse]

Un conteneur `api` passe `unhealthy` après une fuite de connexions à la base. Avec Podman rootless seul (sans Compose ni Kubernetes), que se passe-t-il ? Que fait Docker dans la même situation ? Quelle option Podman permet une réaction automatique, et pourquoi un orchestrateur reste-t-il la vraie réponse ?

Question 6 [Prédiction]

```
podman run --rm --read-only --cap-drop ALL --user 1000 --security-opt no-new-privileges --
entrypoint sh api:1.0 -c 'id; touch /app/x; touch /tmp/x; java -version'
```

Prédisez la sortie ligne par ligne, puis dites ce qu'il manque à cette commande pour qu'une vraie API Spring Boot démarre (indice : Tomcat et la JVM écrivent quelque part).

Question 7 [Compréhension]

Expliquez pourquoi `chown nobody /etc/hostname` réussit dans un conteneur `alpine` lancé normalement mais échoue avec `Operation not permitted` sous `--cap-drop ALL`, alors que dans les deux cas le processus est `root` (UID 0) dans le conteneur. Que révèle cet exemple sur la différence entre « être root » et « avoir des privilèges » ?

Question 8 [Analyse]

La CI d'une équipe pousse chaque build sous `api:latest` et les serveurs font `podman pull api:latest` puis redémarrent. Citez trois incidents que cette stratégie rend possibles, puis décrivez la stratégie de tags recommandée et ce que le déploiement **par digest** garantit de plus que le déploiement par tag de version.

Question 9 [Diagnostic]

```
$ podman push localhost:5000/api:1.0
Error: … pinging container registry localhost:5000: Get "https://localhost:5000/v2/": http:
server gave HTTP response to HTTPS client
```

Expliquez la cause, donnez la correction pour un test local et celle pour un registry d'entreprise, et dites pourquoi la première ne doit jamais devenir la seconde.

Question 10 [Analyse]

Après un `podman push --tls-verify=false localhost:5000/api:1.0`, puis `podman rmi localhost:5000/api:1.0` et `podman pull --tls-verify=false localhost:5000/api:1.0`, `podman images` montre `localhost/api:1.0` et `localhost:5000/api:1.0` avec le **même** `IMAGE ID`. Expliquez ce que le registry a réellement reçu et renvoyé, pourquoi l'ID est identique, et ce que `podman image inspect --format '{{.Digest}}'` ajoute comme garantie.

Question 11 [Diagnostic]

Un collègue lance `podman system prune -a --volumes -f` sur un agent de CI « pour faire de la place » pendant que la stack de tests tourne. Listez ce qui est supprimé et ce qui ne l'est pas, les conséquences sur le build suivant, et la séquence qu'il aurait dû exécuter.

Question 12 [Analyse]

Voici un Quadlet :

```
[Container]
Image=localhost/api:1.0
PublishPort=18080:8080
EnvironmentFile=%h/labo-docker/09/api.env
[Install]
WantedBy=default.target
```

Expliquez ce que `systemctl --user daemon-reload` et `systemctl --user enable --now api` produisent, pourquoi ce service survit à `wsl --shutdown` là où `--restart=always` échoue, et ce qu'il faut encore configurer sur WSL pour que le service démarre sans ouvrir de session.

Question 13 [Analyse]

Décrivez un pipeline CI/CD complet pour `api`, de `git push` à la production, en cinq étapes, en précisant à chaque étape : la commande ou l'outil, ce qui bloque le pipeline en cas d'échec, et quel labo de cette série a introduit la notion. Où se placent le scan d'image, le `HEALTHCHECK`, les limites de ressources et le digest ?